

Multi-Agent Ecosystems: Part - 1



Agentic RAG and Multi-Agent Frameworks



Engage and Think



Imagine you're a product lead tasked with rolling out an AI-based onboarding agent for a financial services platform.

You decide to deploy a single **super-agent** capable of answering questions, verifying documents, and escalating cases.

Two weeks later, the system starts auto-approving risky clients, skipping critical steps, and crashing during handoffs.

How might breaking this workflow into specialized agents reduce failure and improve control?

Learning Objectives

By the end of this lesson, you will be able to:

- 🔗 Identify where traditional RAG systems fall short in handling complex tasks
- 🔗 Make use of agentic RAG ideas to spot and fix common problems in multi-step AI workflows
- 🔗 Analyze the distinct functions of planner, retriever, memory, and tool-executing agents in agentic systems
- 🔗 Create basic agentic system designs based on real product goals and task types
- 🔗 Evaluate agentic AI frameworks such as CAMEL, AutoGPT, and MetaGPT to determine the best fit for different use cases

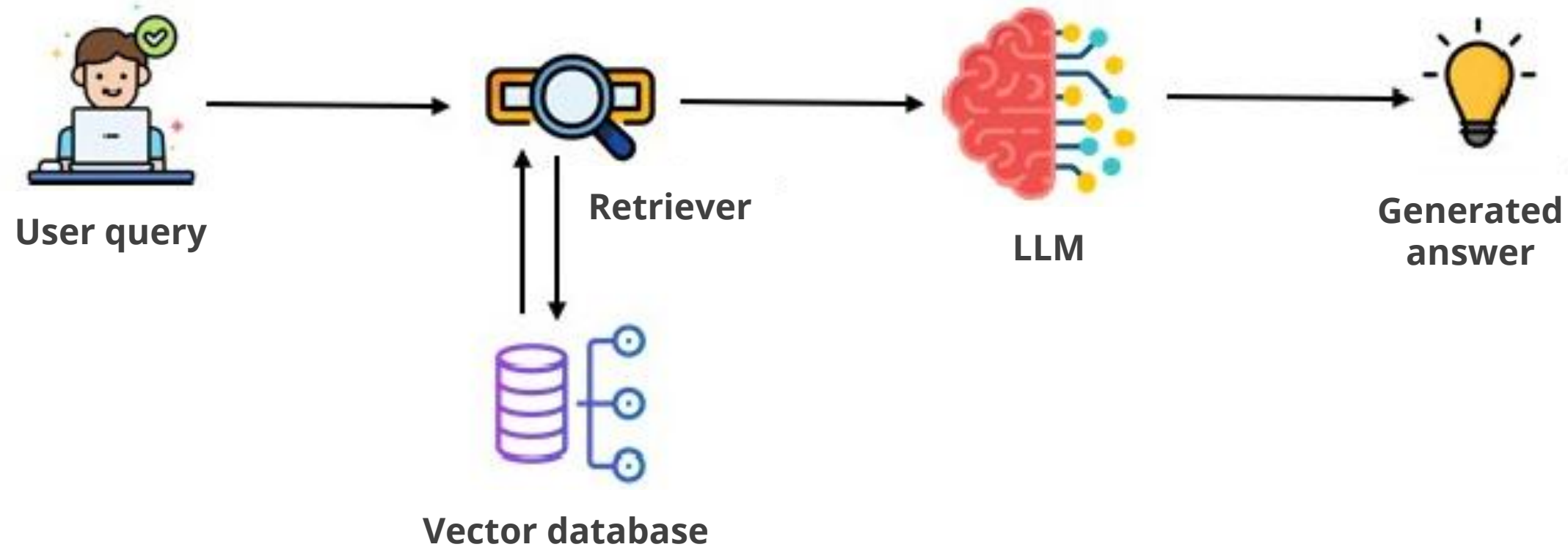




Introduction to RAG and Agentic RAG

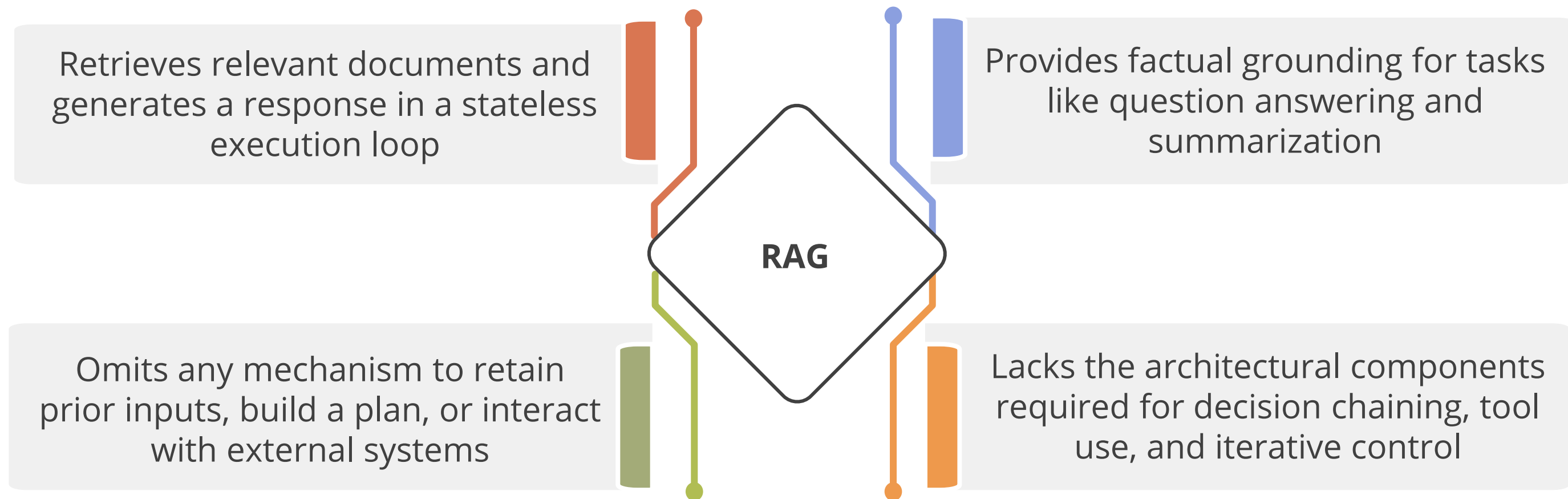
Traditional RAG Systems

Traditional Retrieval-Augmented Generation (RAG) systems retrieve relevant context from a knowledge base and pass it to a language model to generate grounded responses for a single-step query.



Traditional RAG: Strengths and Limitations

RAG grounds LLM outputs via external retrieval, but its stateless, single-pass loop limits planning, memory, and multi-step orchestration.



Stateless execution loop: Each query runs independently, without memory of prior context.

Where Traditional RAG Falls Short: Example

When RAG works:

Input: *What are the documentation requirements for new users?*



RAG system



Output: *New users must upload a valid ID and proof of address.*

The answer is factually correct, and the task is complete.

When RAG breaks down:

Input: *Please check if this user has uploaded all required KYC documents and escalate if any are missing*



RAG system

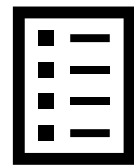


Output: *Users must upload a valid ID and proof of address.*

The system does not verify document submission, make approval or escalation decisions, or track state across steps.

Extending RAG with Agentic Capabilities

Agentic RAG extends the traditional retrieval pipeline by incorporating memory, planning, and tool enabling systems to reason, adapt, and act over multiple steps.



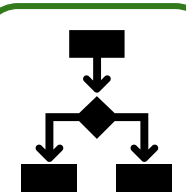
It introduces a task planner to break down goals into smaller executable steps.



It integrates memory to retain context across inputs, interactions, and stages.



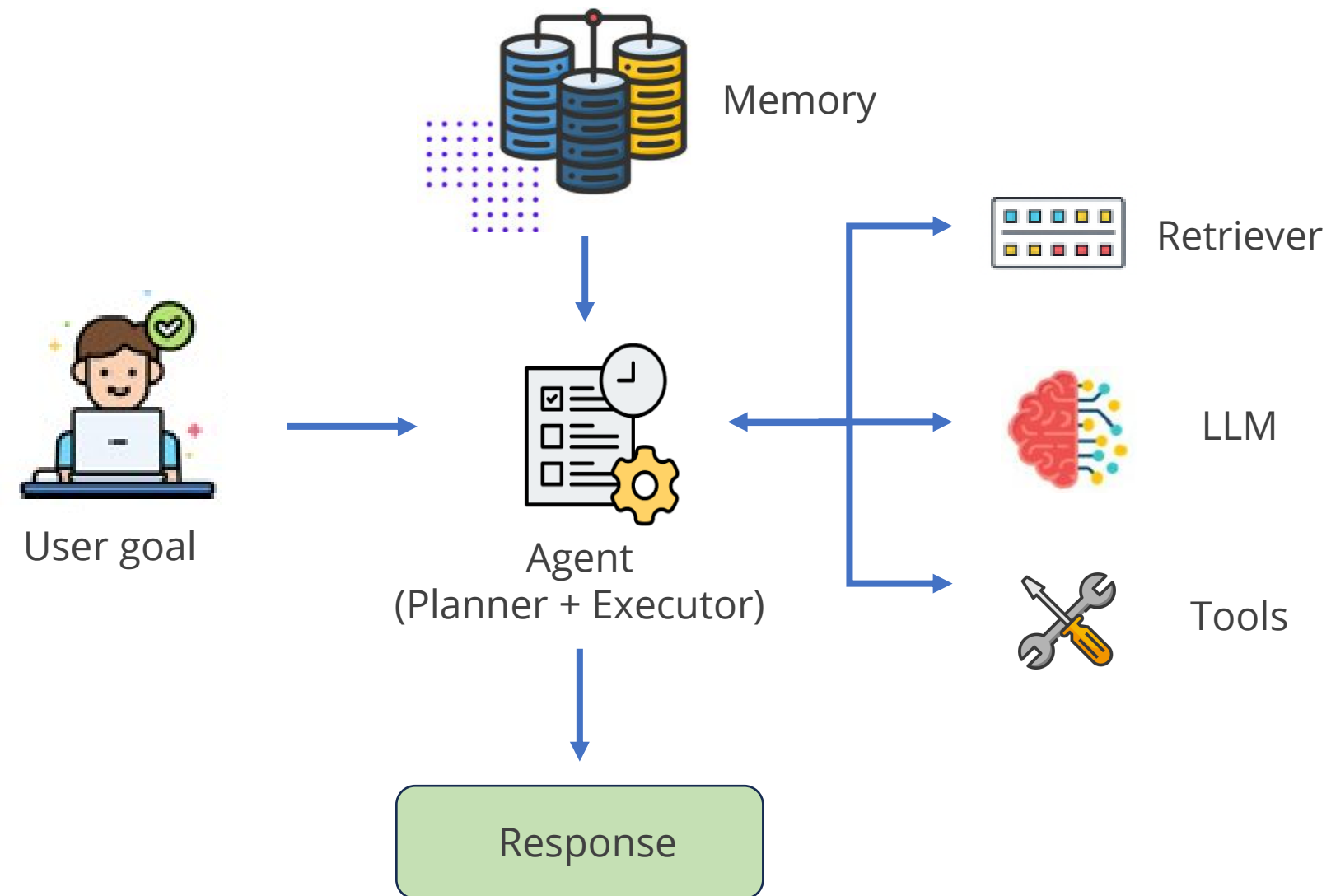
It enables tool use and external API calls to interact with the world beyond text.



It supports dynamic decision-making and control flow within multi-step workflows.

Anatomy of Agentic RAG

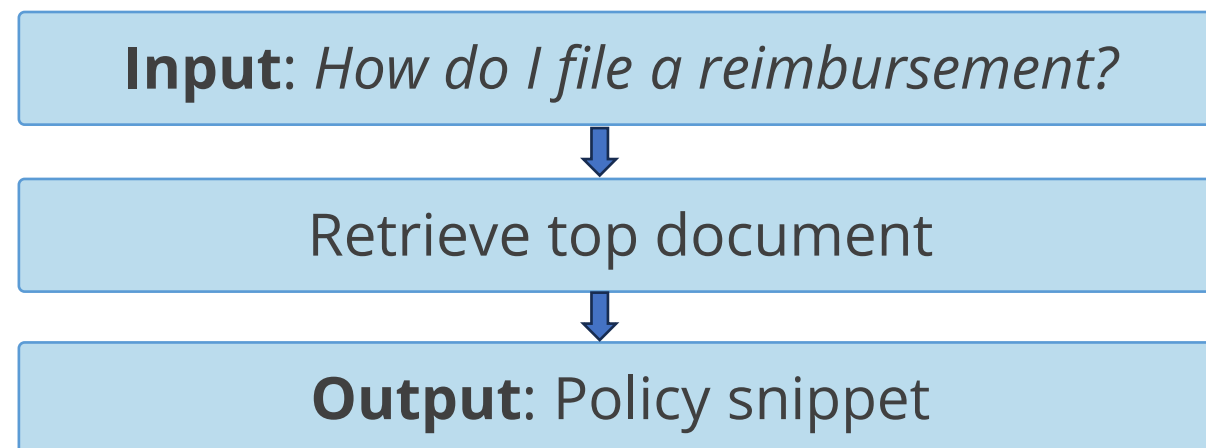
The Agent receives a goal, consults memory, and plans multi-step actions by invoking retrieval, generation, and tools, driving iteratively toward a complete system response.



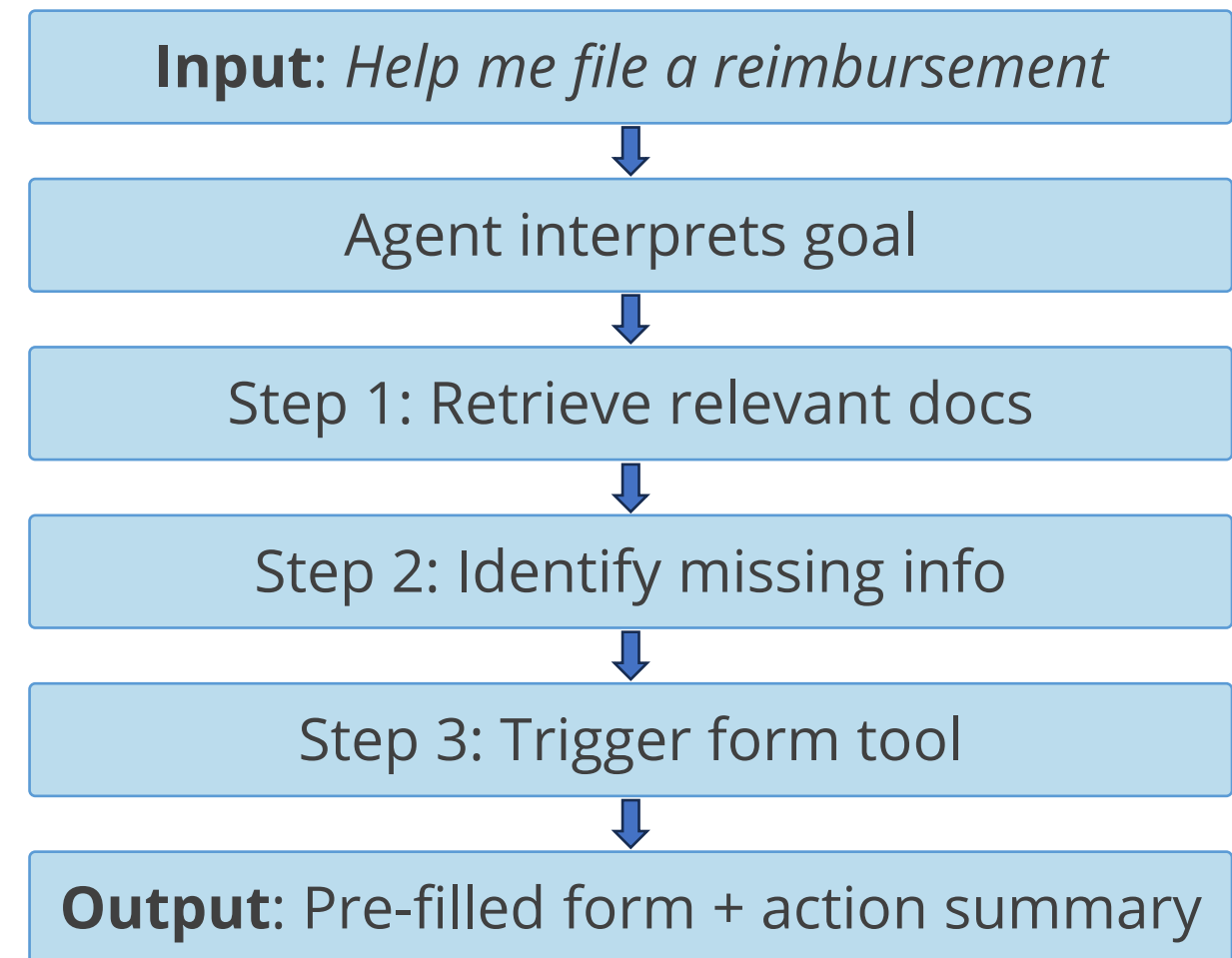
Task-Oriented Execution with Agentic RAG

Agentic RAG frameworks enable dynamic task execution, unlike traditional RAG, which stops at retrieval. The comparison below shows how agent-based systems resolve goals through multi-step orchestration.

Traditional RAG



Agentic RAG



Memory Strategies in Agentic RAG

Agentic RAG systems use memory to track progress, maintain context, and recall prior actions, enabling multi-step reasoning. The table below highlights two key memory types that support this capability.

Memory type	Purpose	Examples	Usage in Agentic RAG
Episodic memory	Captures short-term task state	In-memory cache, Redis, short-term vector store	Tracks steps completed, tools used, and recent decisions
Long-term memory	Stores reusable knowledge across sessions	FAISS, Pinecone, Weaviate, and Elasticsearch	Retains facts, user preferences, past task patterns, and outcomes

Integrating Tools in Agentic RAG Systems

As agents move from answering to acting, they begin invoking tools, APIs, databases, or automation endpoints to complete tasks. Tool integration reshapes how agentic systems behave under real-world constraints.

Enables real-world action

Agents use tools to fetch data, trigger workflows, or complete user tasks.

Introduces system-level risk

Every call adds latency, variability, and potential for failure.

Demands structured inputs and parsing

Agents must adhere to API formats and handle outputs precisely.

Requires traceability and logging

Each tool invocation should be observable for debugging and auditing.

Use Case: Traditional vs. Agentic RAG in Legal Reviews

Scenario: A legal tech platform must review 500 contracts daily to identify and flag risky clauses, ensure compliance, and escalate any ambiguities.

Here’s how Traditional RAG and Agentic RAG differ in supporting this workflow:

Traditional RAG	Agentic RAG
Extracts isolated clauses from legal documents	Plans and executes end-to-end document review flows
Treats each request as a one-time lookup	Retains task memory across steps and sections
	Flags risks and routes complex cases for escalation
	Iterates through subtasks to ensure policy coverage

Strategic Challenges in Agentic RAG

Deploying Agentic RAG surfaces critical challenges in control, latency, and accuracy, each influencing rollout and adoption decisions. The table outlines what product leaders must weigh:

Dimension	Agentic challenge	PM consideration
Control	Agents plan and act autonomously across multiple steps.	Define clear boundaries, fallback rules, and approval thresholds.
Latency	Multi-step reasoning increases response time.	Optimize workflows to balance depth of reasoning with user experience.
Accuracy	Outputs emerge from dynamic decision-making rather than fixed logic.	Invest in verification layers, observability, and continuous evaluation.

Note: These challenges directly affect user experience, compliance, and scalability; they are not technical side effects.

Quick Check



Why does Agentic RAG outperform traditional RAG in task orchestration?

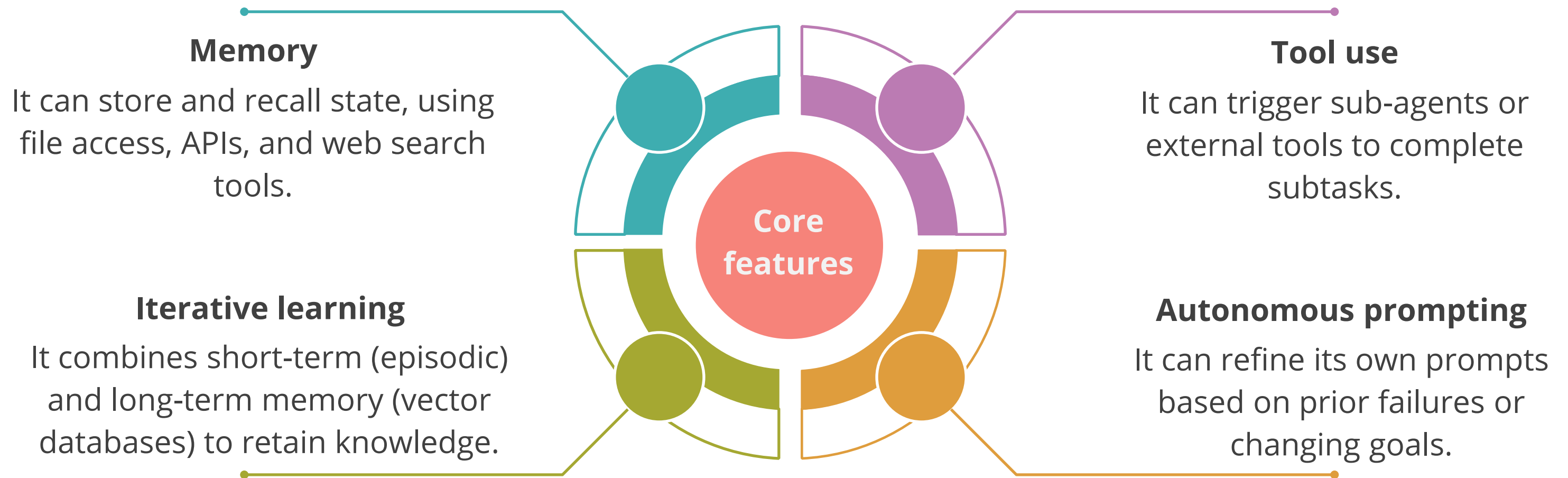
- A. It uses decision trees.
- B. It relies on rule-based AI.
- C. It dynamically plans and tracks progress.
- D. It ignores context to boost speed.



Essential Agentic RAG Designs

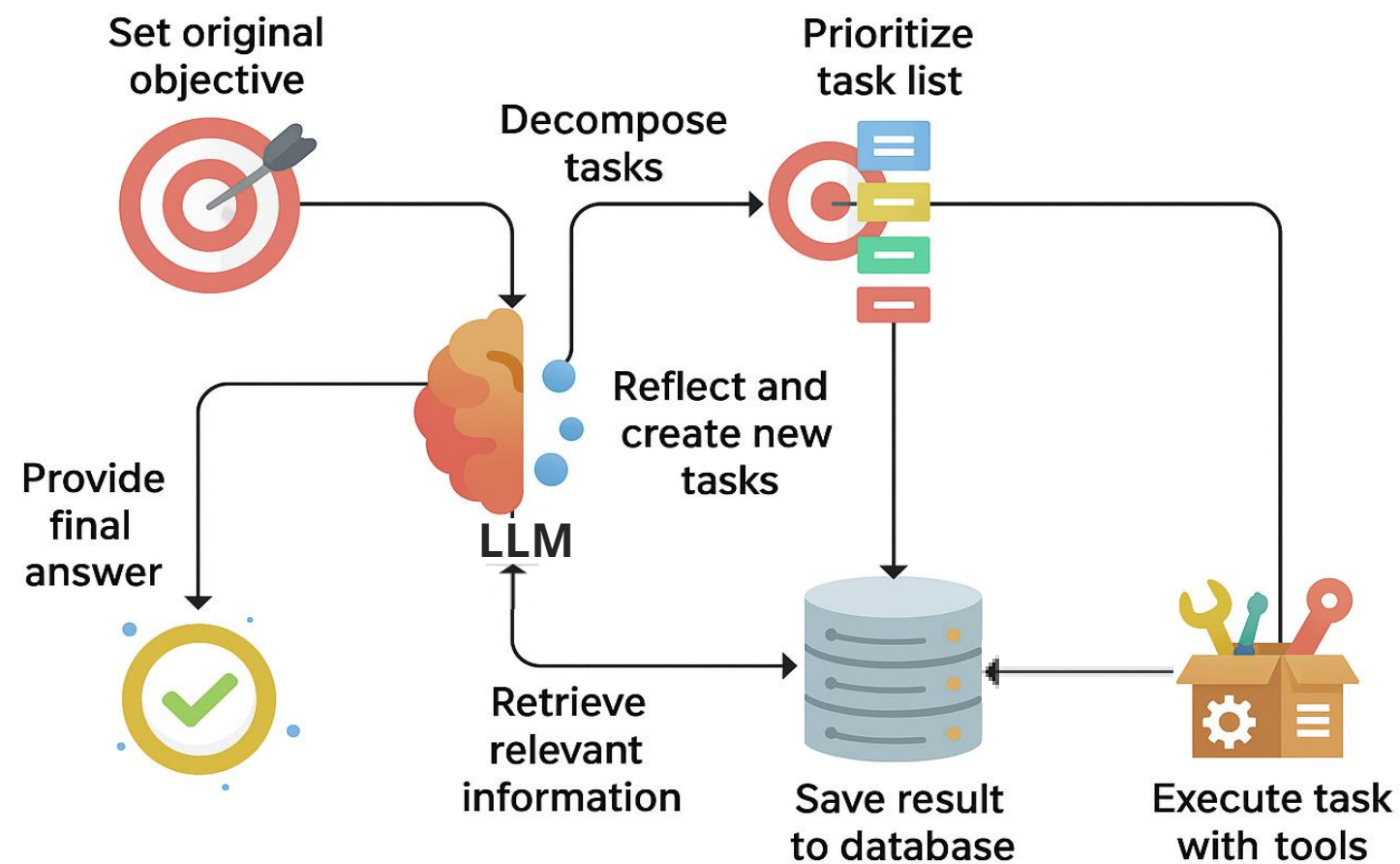
AutoGPT: Recursive Task Decomposer

It uses a looped planning-execution architecture to autonomously break down broad user goals into smaller subtasks, retrieving information and taking actions in a sequence until the objective is met.



Unlike RAG, which retrieves once per query, AutoGPT plans, remembers, and adapts until the goal is met.

AutoGPT framework: Overview

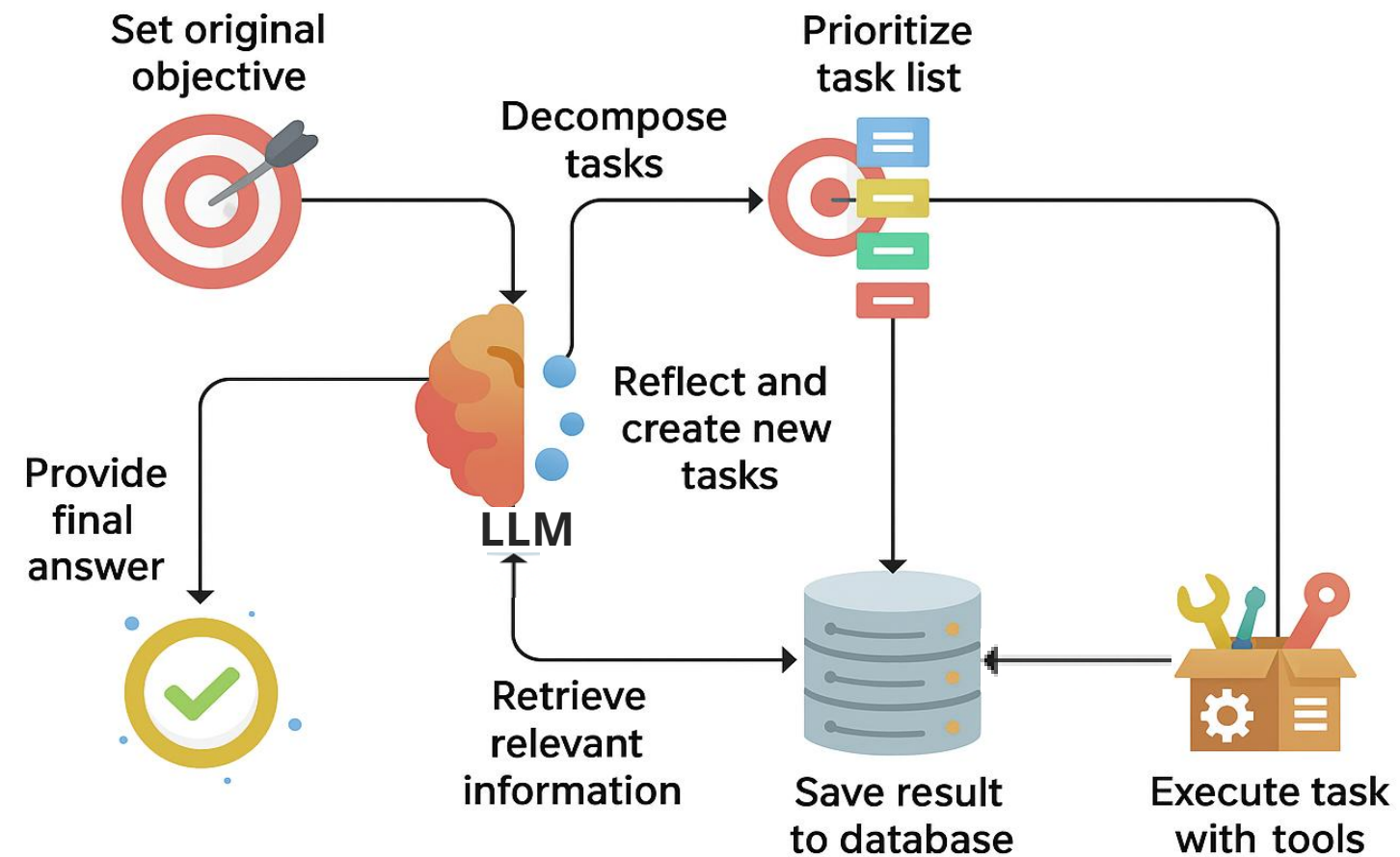


AutoGPT framework

Step-by-step process:

- 1. Set original objective:** Defining the high-level goal to be achieved
- 2. Decompose tasks:** Breaking down the goal into smaller, manageable subtasks
- 3. Prioritize task list:** Ordering tasks based on relevance or sequence
- 4. Execute task with tools:** Performing actions with the help of external tools

AutoGPT framework: Overview



AutoGPT framework

5. **Save result to database:** Storing outputs and intermediate data for reuse
6. **Retrieve relevant information:** Pulling necessary data to support ongoing tasks
7. **Reflect and create new tasks:** Reviewing outcomes and dynamically updating the task list
8. **Provide final answer:** Delivering the complete result once the objective is met

AutoGPT Framework: Benefits



Automates goal decomposition

Breaks complex objectives into actionable subtasks without manual planning



Enables continuous task refinement

Reflects on progress and adjusts the plan dynamically for better outcomes



Improves execution efficiency

Uses tools and APIs to complete tasks autonomously and at scale



Enhances adaptability

Responds to new information and changing goals in real time



Supports modular integration

Connects with external systems for task execution, data retrieval, and storage

AutoGPT Framework: Use Case

Scenario: A product team at a software as a service (SaaS) company wants to improve its collaboration tool based on user feedback.

How AutoGPT helps:

Sentiment mining

AutoGPT analyzes reviews, support tickets, forums, and social platforms to extract common feedback.

Example:

- *Mobile UI is confusing.*
- *Need for dark mode support*

Prototyping assistance

AutoGPT assists with debugging, UI wireframes, and onboarding drafts.

Example actions:

- *Fix sync issue across devices*
- *Draft dark mode toggle layout*
- *Write onboarding walkthrough*

Trend aggregation

AutoGPT clusters similar feedback and ranks it by frequency and urgency to identify dominant themes.

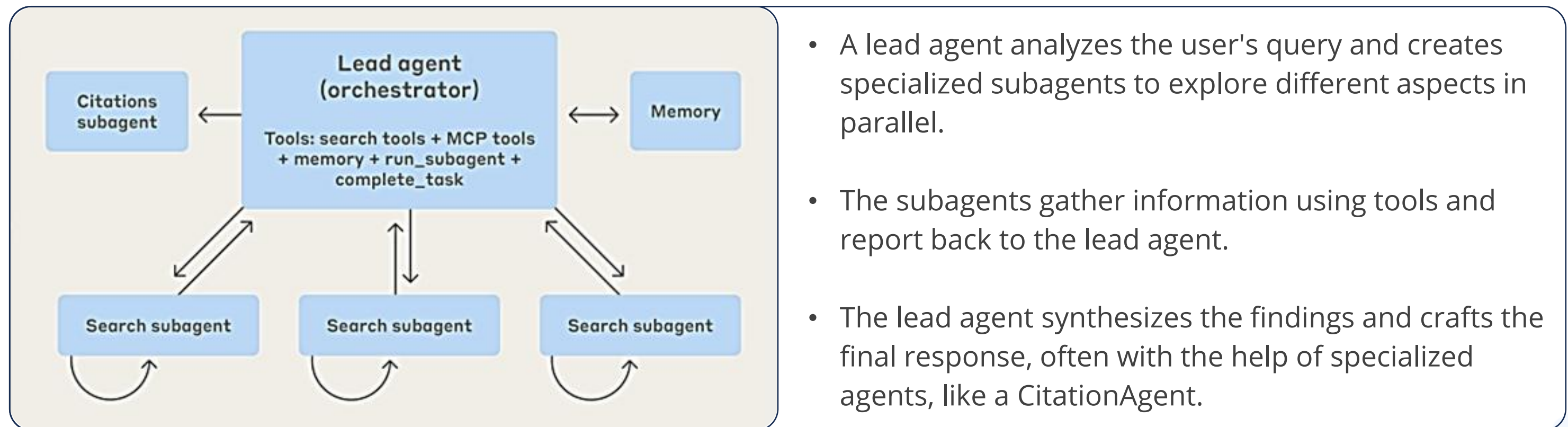
Example outputs:

- *35% of users request dark mode.*
- *20% cite mobile UI confusion.*
- *15% request better search filters.*

Anthropic Research Agent: Overview

Anthropic AI agents are AI systems built upon Anthropic's LLMs, like the Claude series, designed to go beyond simple responses and perform complex tasks autonomously.

How Anthropic's multi-agent system works:



Key idea: Parallel processing by specialized agents enhances efficiency and depth of research.

Anthropic Research Agent: Use Case

Scenario: A researcher requires assistance in analyzing the challenges of implementing climate policy in India.

Input: *What are the challenges of climate policy implementation in India?*



Agentic workflow:

Subagent 1
(Query planner)

Splits into subtopics:
policy overview,
barriers, stakeholder
perspectives



Subagent 2
(Search agent)

Searches government
reports, peer-
reviewed studies, and
news coverage



Subagent 3
(filter agent)

Filters contradictions,
checks for weak or
biased sources



Subagent 4
(Synthesis agent)

Combines all findings
into a balanced,
coherent summary



Output: Delivers a research-ready report with key barriers, evidence-backed insights, and properly cited sources

Demo: Understanding the Anthropic Research Agent



Duration: 10 minutes

Overview: This demo illustrates how Anthropic's Research Agent inside Claude simulates multi-agent research workflows. The lead agent plans and orchestrates tasks, spawns specialized subagents, maintains memory across steps, and synthesizes results with citations. By following this simulation, you will gain an understanding of how complex research questions are decomposed, processed in parallel, and reassembled into coherent outputs, preparing them for real-world agentic AI applications.

DEMONSTRATION

Quick Check



How does AutoGPT handle complex goals?

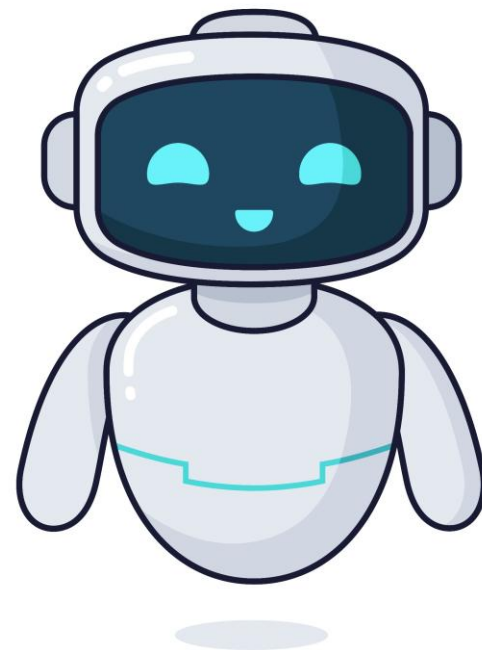
- A. It uses a manual tagging system to control agent behavior.
- B. It retrieves documents and presents static outputs.
- C. It decomposes goals and updates task lists iteratively.
- D. It executes subtasks using human-in-the-loop supervision.



Introduction to Agentic Systems

Single-Agent System: Overview

It is an architecture in which one intelligent agent is responsible for completing all tasks in a given environment. This agent acts independently, handling decisions, inputs, and actions without coordination.

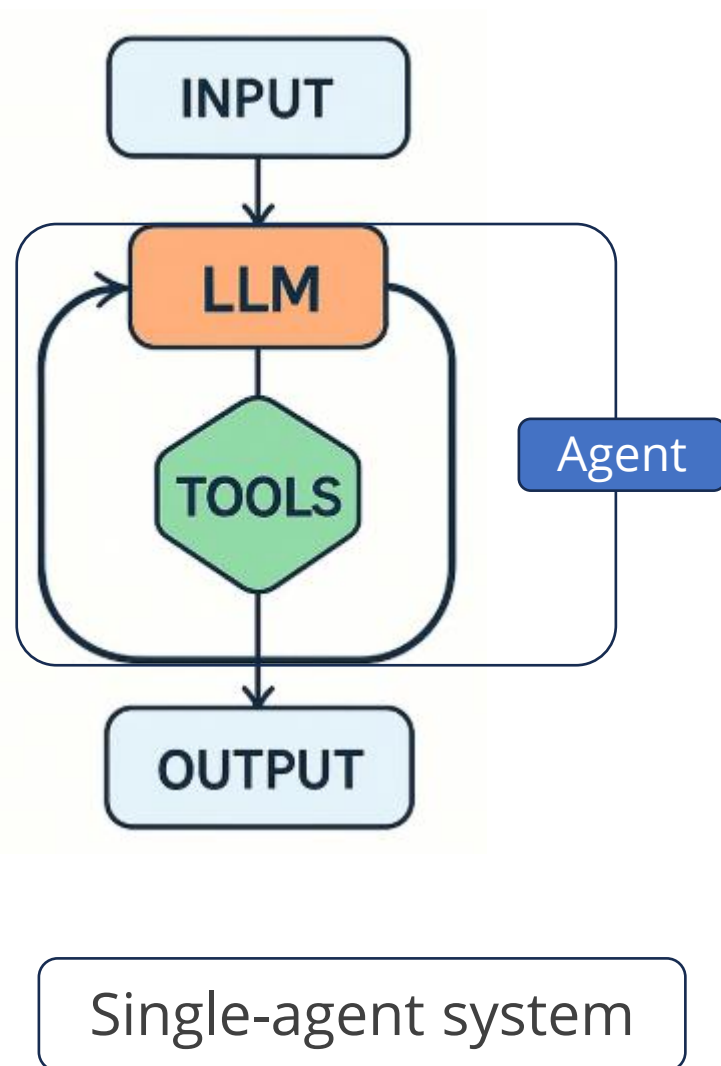


Note

It is ideal for linear or contained workflows where task complexity is limited and cross-functional coordination is unnecessary.

Single-Agent System: Architecture

A single-agent system solves a task end-to-end through one autonomous agent without delegating to other agents.



01

Agent receives the task input

Accepts a goal or instruction to begin execution

02

Agent executes reasoning and actions

Plans and performs steps independently to resolve the task

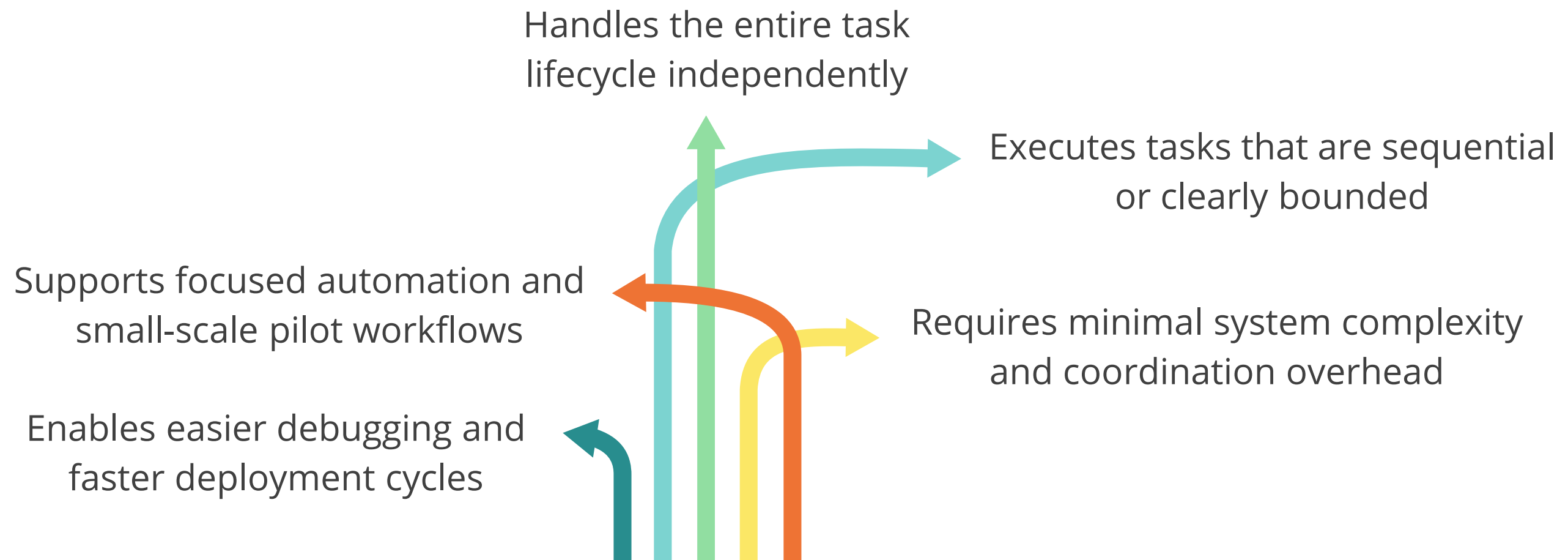
03

Agent delivers the final output

Produces a solution directly without any inter-agent collaboration

Single-Agent System: Characteristics

A single-agent system is best understood through the following core attributes:



Single-Agent System: Benefits

Single-agent systems are ideal for testing intelligent workflows before adding complexity.
The key advantages they offer in the early-stage automation are:

Makes early automation easier by
focusing on small, manageable tasks

Minimizes risk during
testing and deployment

Provides a stable foundation for
future scaling

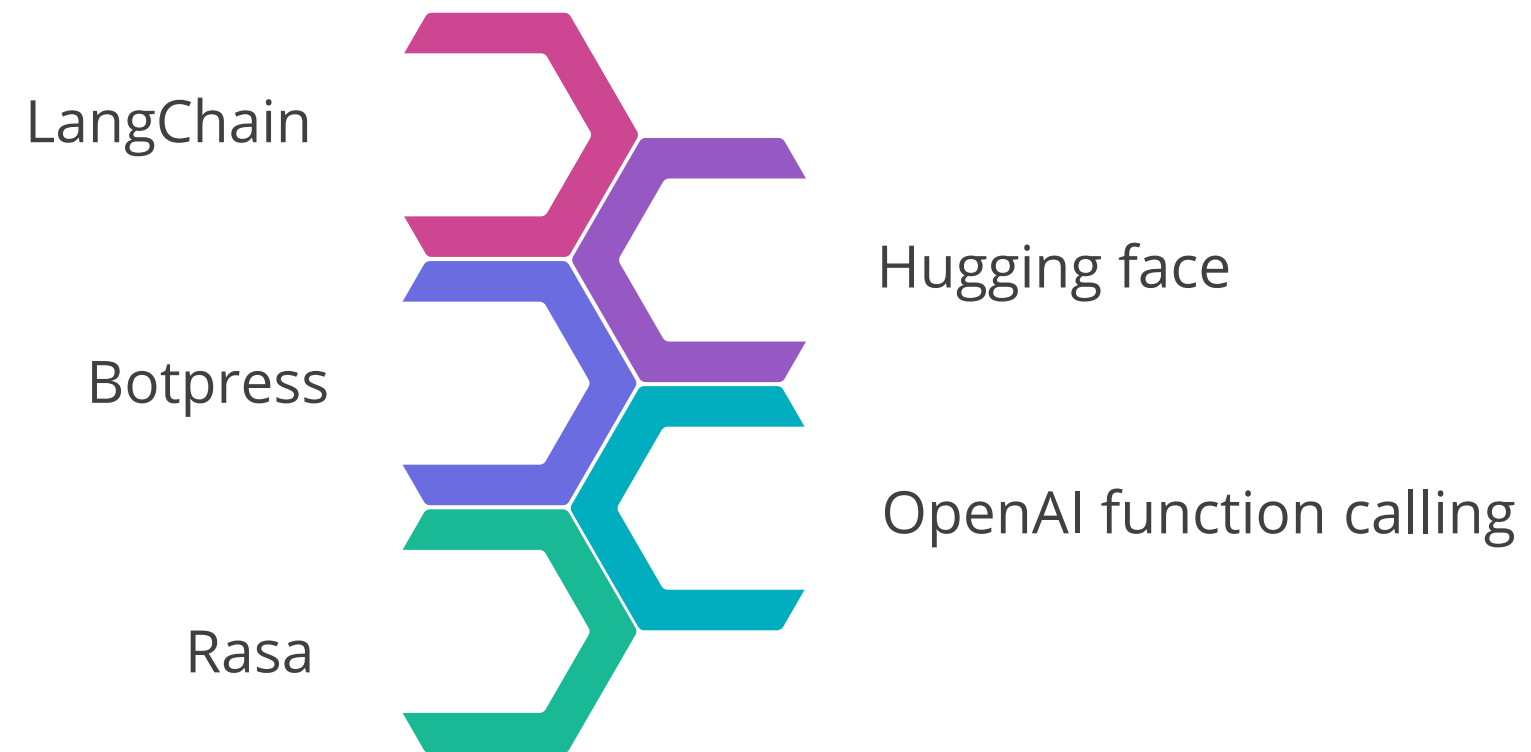
Supports routine tasks that do not
require collaboration

Reduces development time and cross-
team dependencies



Single-Agent System: Tools

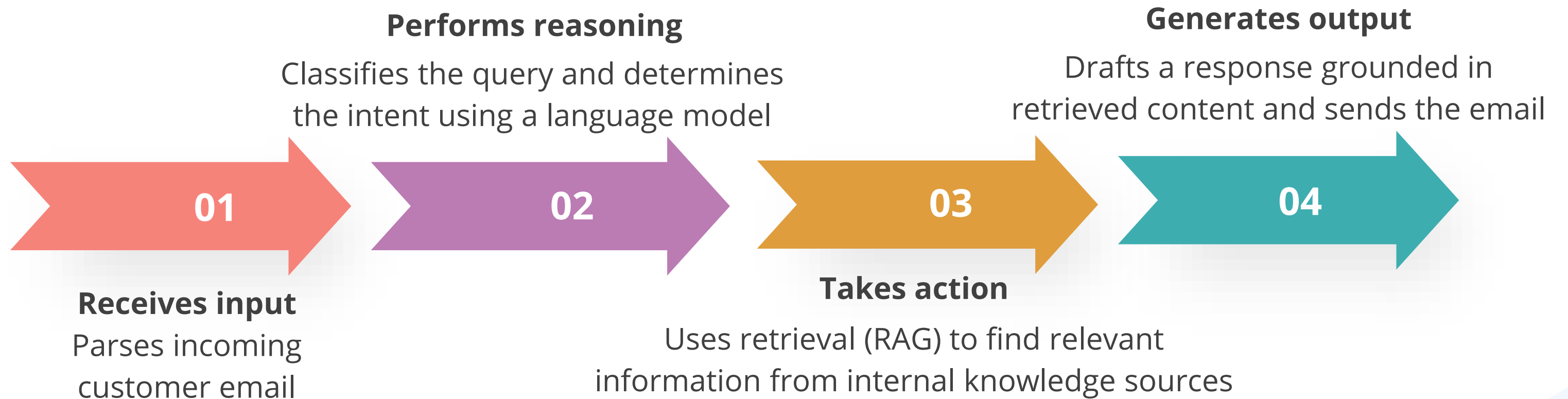
Below are some tools designed to build single-agent systems that manage tasks independently.



Single-Agent System: Use Case

Scenario: A customer support team wants to automate replies to frequently asked email queries about shipping, returns, and order status.

Workflow of a single-agent system:



This single-agent system completes the full loop from perception to action without delegating to other agents.

Quick Check

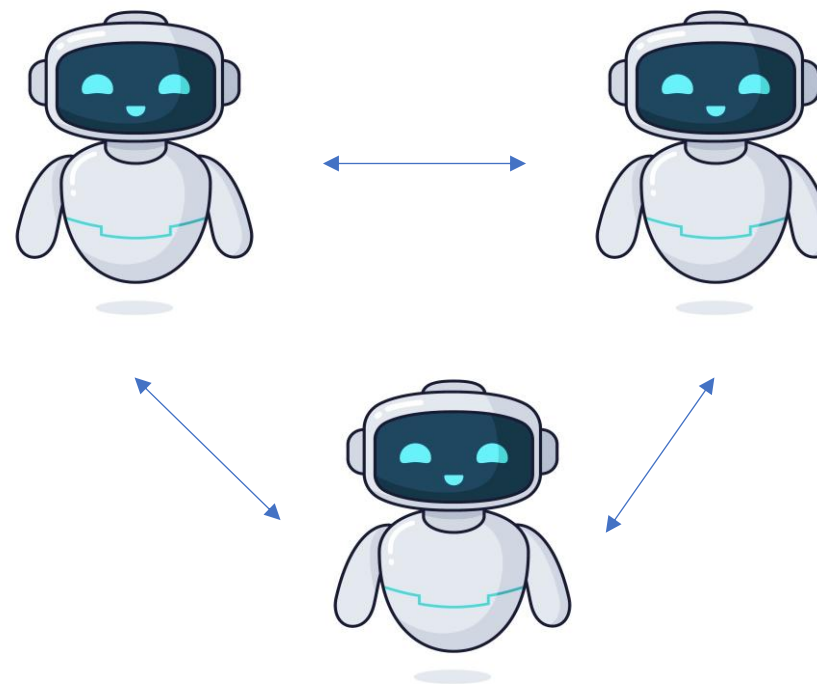


What is a defining characteristic of a single-agent system?

- A. It requires multiple agents to collaborate on subtasks.
- B. It performs all reasoning and actions independently.
- C. It is ideal for large-scale distributed workflows.
- D. It relies on supervisor agents for planning.

Multi-Agent Systems: Overview

It consists of two or more intelligent agents that work together to complete a task or set of tasks. Each agent has a defined role, specific goals, and often its own tools or data sources.

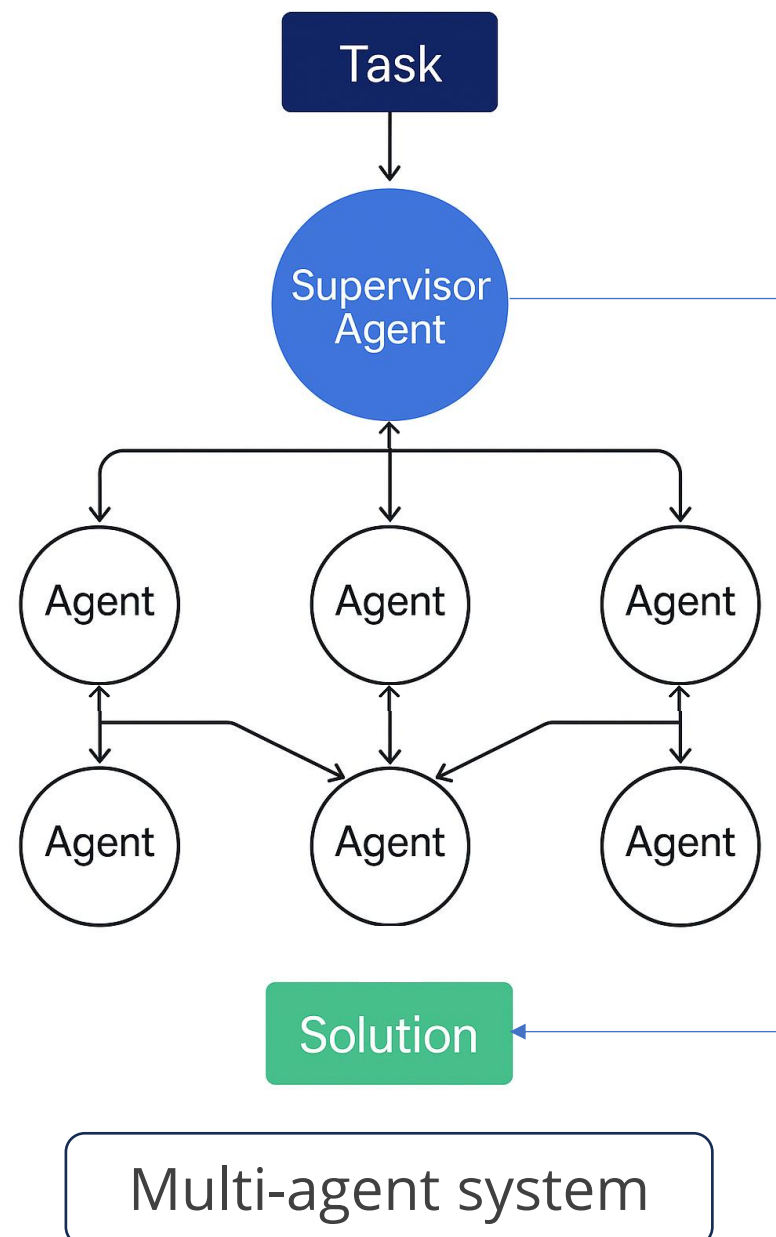


Note

They are ideal for complex workflows that require specialization, parallel processing, and inter-agent collaboration.

Multi-Agent Systems: Architecture

A multi-agent system distributes tasks across multiple specialized agents, coordinated by a supervisor agent to achieve complex outcomes.



01

Supervisor agent receives the task

Assigns subtasks to specialized agents based on their roles

02

Agents execute in parallel

Perform distinct actions with contextual awareness

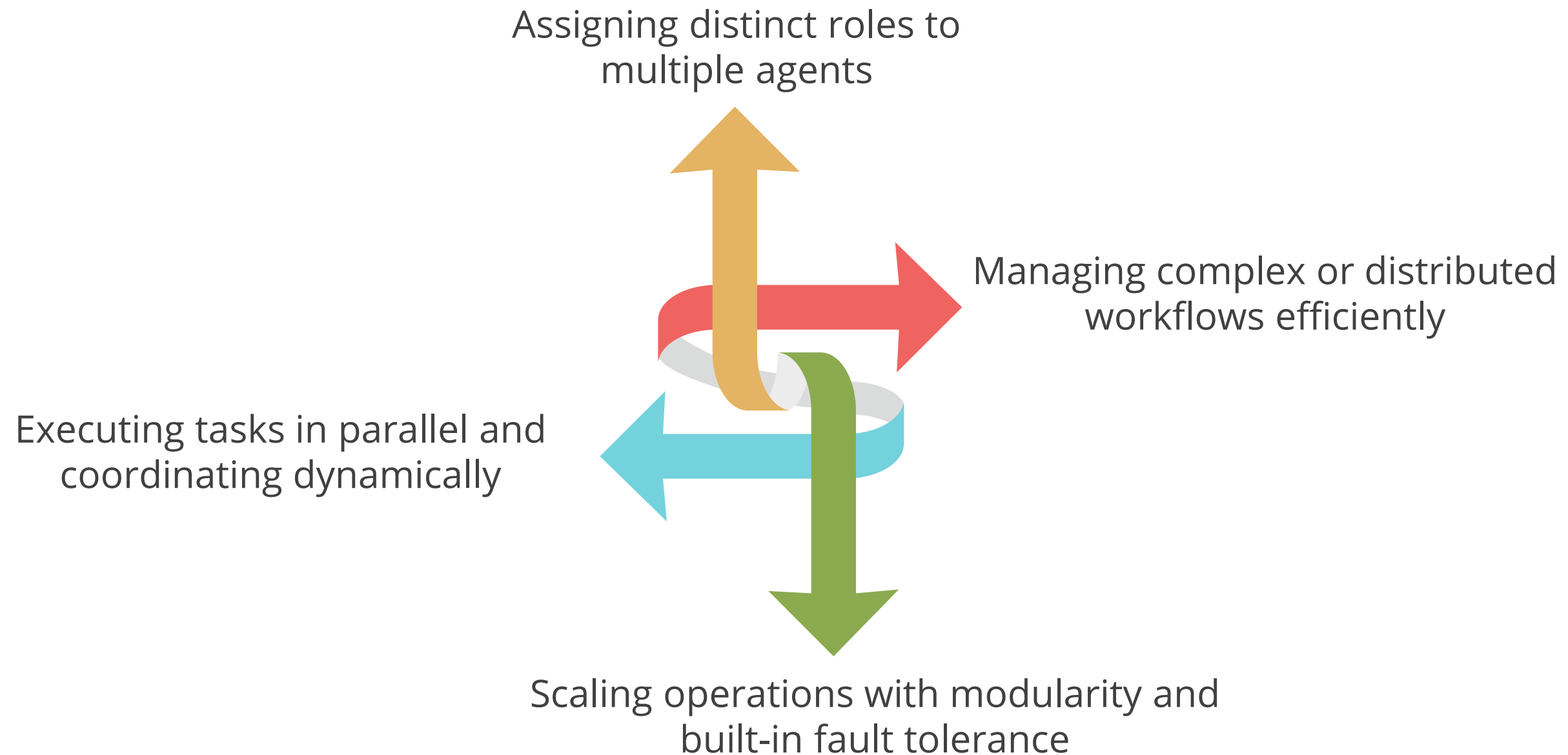
03

Final output is synthesized

Combines agent results to deliver an end-to-end solution

Multi-Agent Systems: Characteristics

Multi-agent systems are built to support complex, collaborative operations through the following features:



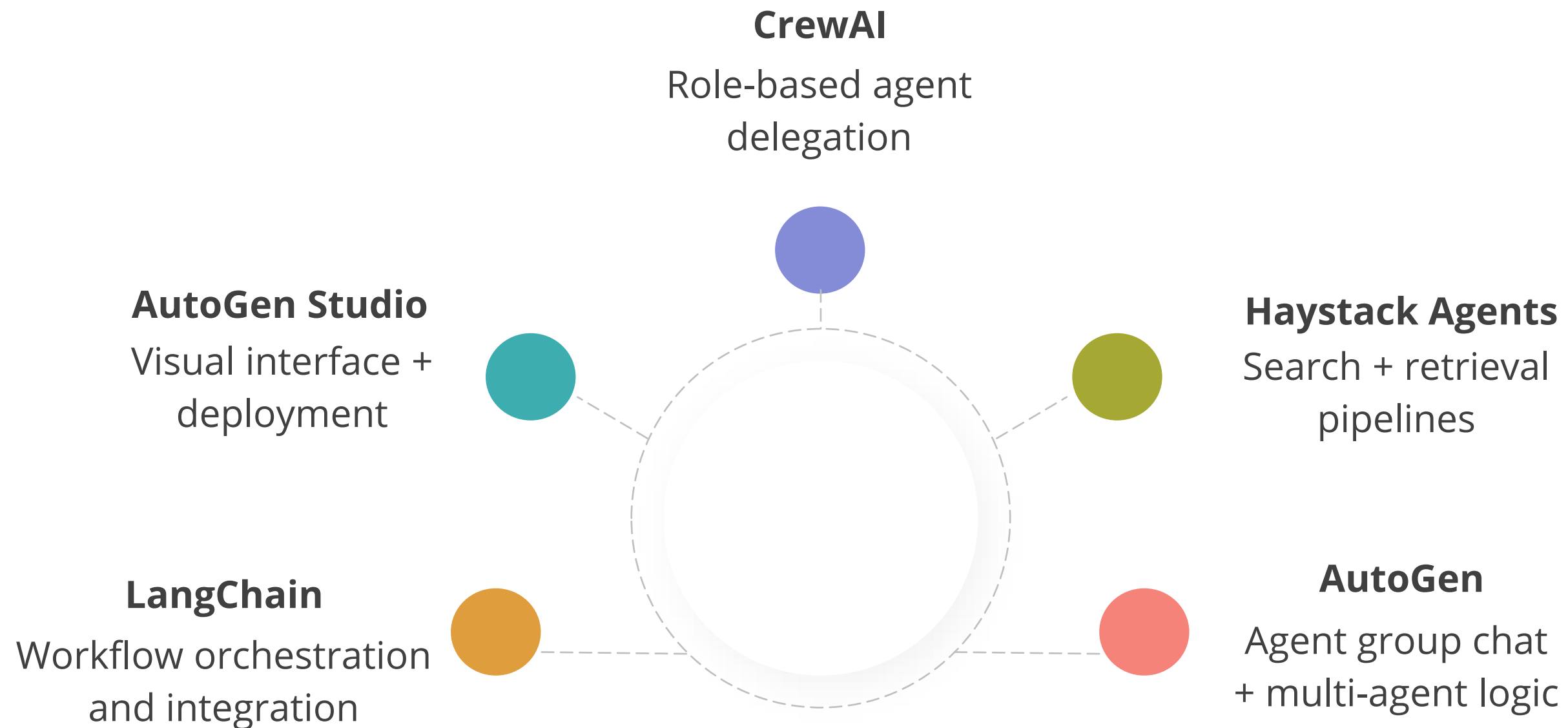
Multi-Agent Systems: Benefits

These systems mimic real-world team dynamics by assigning distinct responsibilities to specialized agents, leading to more efficient and flexible execution. The key benefits are as follows:



Multi-Agent Systems: Tools

The tools below help implement multi-agent frameworks by offering orchestration, memory handling, and agent collaboration infrastructure.



Multi-Agent Frameworks

They enable scalable coordination across specialized agents. The following are three widely used multi-agent frameworks.



CAMEL

Simulates structured conversations between role-based agents to co-develop solutions through iterative negotiation



AgentVerse

Provides a modular sandbox where agents interact with diverse tools and memory types for controlled experimentation



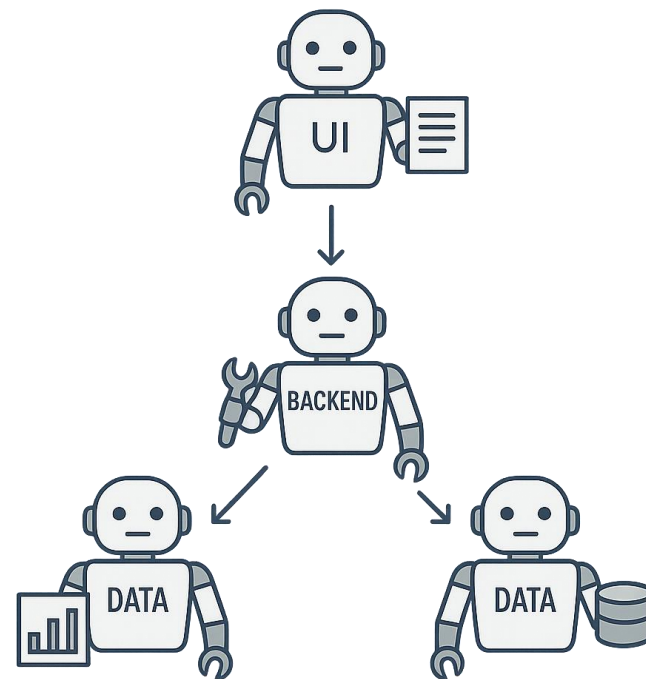
MetaGPT

Emulates real-world team workflows by assigning predefined roles (PM, Engineer, QA) and coordinating via standard procedures

Multi-Agent Systems: Use Case

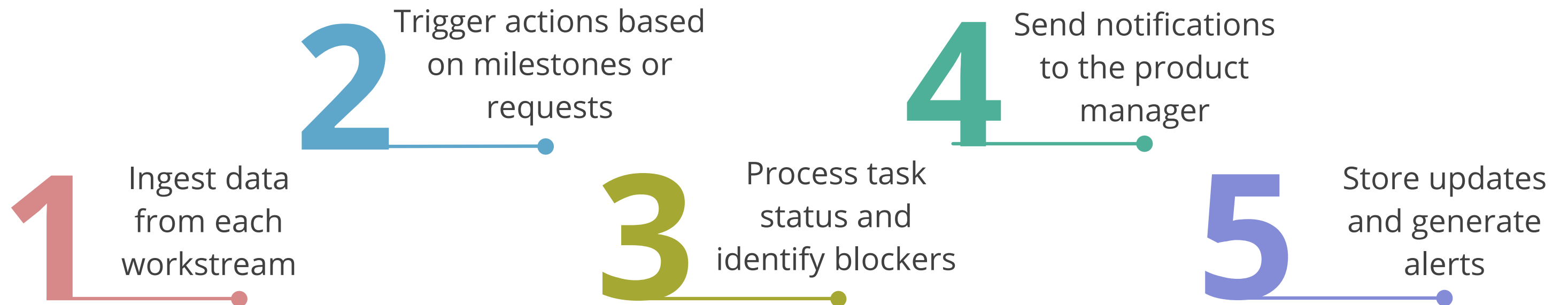
Use Case: Coordinating multiple product workstreams

Scenario: A product leader managing a platform rollout wants to replace a bottlenecked project manager with specialized agentic workflows, where teams like UI, backend, and data agents automate specific tasks and collaborate autonomously.



Multi-Agent Systems: Use Case

Solution: Rather than relying on manual coordination, the leader sets up multiple AI agents to manage each workstream. These agents automate the following tasks:



Single-Agent vs. Multi-Agent System

The key differences between single-agent and multi-agent architectures are as follows:

Single-agent system	Multi-agent system
Handles all tasks with one agent	Distributes tasks across multiple agents
Operates without inter-agent collaboration	Collaborates dynamically between agents
Runs simple, linear workflows	Manages complex, distributed workflows
Scales with limitations	Scales easily by adding agents
Fails entirely if the agent fails	Continues operations through agent redundancy

Quick Check



Which of the following statements is true about single-agent and multi-agent systems?

- A. Single-agent systems always scale better than multi-agent systems.
- B. Multi-agent systems avoid all communication overhead.
- C. Single-agent systems are less fault tolerant but easier to manage.
- D. Multi-agent systems require fewer resources to deploy than single-agent systems.

Guided Practice



Overview

Duration: 20 minutes

You are a product manager at a tech startup building a web-based habit tracking app. Your goal is to improve user retention by introducing a gamification feature. In this guided practice, you will use MetaGPT's multi-agent ecosystem to generate a complete product specification. By providing a business prompt, MetaGPT will allocate tasks across roles such as Product Manager, Designer, and Engineer to deliver structured outputs. You will log in, run the workflow, analyze the agent collaboration, and review the generated specification. By completing this practice, you will learn how to operate MetaGPT effectively, observe how multiple agents collaborate in structured product development, and evaluate how prompt engineering shapes AI-generated outputs.

GUIDED PRACTICE

Key Takeaways

- Agentic RAG combines retrieval with planning and evaluation, improving relevance and adaptability of retrieved content.
- AutoGPT autonomously decomposes goals, prioritizes tasks, uses tools and memory, and refines actions through self-reflection.
- Anthropic's Research Agent uses a lead agent to coordinate specialized subagents, improving research depth, speed, and citation accuracy.
- Single-agent systems are ideal for structured, linear tasks and fast deployments but lack flexibility for complex workflows.
- Multi-agent systems distribute work across specialized agents under a supervisor, enabling parallel execution, modular scaling, and coordinated synthesis of complex outputs.



Q&A

